

Treinamento Incremental aproximado de sistemas incorporados inteligentes

Caio Caracas Veras

Agosto 2026

1 Contextualização

A inferência de uma rede convolucional consiste em somas de produtos, operações cujo volume domina o tempo de processamento. Um circuito dedicado executa muitas delas por ciclo e organiza o movimento de dado conforme a estrutura do problema. Em FPGA, multiplicar e acumular corresponde à função do bloco DSP presente no dispositivo.

O projeto desse circuito depende da rede. Número de camadas, número de filtros e tamanho de kernel determinam quanto de cada recurso cada camada consome, e uma repartição adequada a uma rede não serve para outra. Este trabalho automatiza essa etapa: a partir dos hiperparâmetros da rede treinada, um script reparte os multiplicadores entre as camadas, escreve o Verilog, sintetiza e mede o resultado na placa.

2 Objetivos

Construir uma arquitetura parametrizada para inferência de redes convolucionais 1D em FPGA, derivada automaticamente da rede treinada, e caracterizá-la em silício.

2.1 Objetivos Específicos

- Definir um motor de camada parametrizado que atenda qualquer combinação de canais, filtros, kernel e *pooling*.
- Repartir os multiplicadores entre as camadas de modo que nenhuma se torne gargalo.
- Operar em fluxo contínuo, sem recomputar a janela deslizando a cada decisão.
- Gerar o RTL, sintetizar e medir sem intervenção manual.
- Caracterizar exatidão, determinismo, área, frequência e consumo.

3 Fundamentação Teórica

3.1 Convolução 1D e o bloco DSP

Uma camada convolucional unidimensional produz, para cada filtro de saída f e cada posição x do sinal:

$$y[f][x] = b[f] + \sum_{c=0}^{NIF-1} \sum_{k=0}^{K-1} w[f][c][k] \cdot a[c][x+k] \quad (1)$$

em que a é a entrada com NIF canais, w são os pesos, K é o tamanho do kernel e b a polarização.

As saídas são independentes entre si, de modo que o paralelismo fica limitado pelos recursos disponíveis. O acesso à memória é regular e conhecido em tempo de projeto, pois calcular $y[f][x+1]$ reaproveita $K-1$ das K amostras usadas em $y[f][x]$, reuso que um registrador de deslocamento entrega diretamente.

O bloco DSP48E1 realiza um produto de 25×18 bits seguido de soma de 48 bits, com registradores internos entre as duas etapas. Com operandos de 8 bits, o teto é um produto por bloco por ciclo, e é contra esse teto que a eficiência aritmética é medida.

3.2 Quantização inteira

A rede é convertida para aritmética inteira de 8 bits por escala simétrica, seguindo Wu et al. [1] e Jacob et al. [2]. Um tensor real t é representado por um inteiro q e um fator de escala s :

$$t \approx s \cdot q, \quad q = \text{clamp}\left(\text{round}\left(\frac{t}{s}\right), -2^{b-1}, 2^{b-1} - 1\right) \quad (2)$$

Nos pesos, a escala vem do máximo absoluto do tensor, uma escala por camada. Nas ativações, o máximo é estimado ao longo do treinamento por média móvel exponencial com $\alpha = 0,99$:

$$s_w = \frac{\max |w|}{2^{b-1} - 1}, \quad p_n = \alpha p_{n-1} + (1 - \alpha) \max |x_n|, \quad s_x = \frac{p_n}{2^{b-1} - 1} \quad (3)$$

A quantização ocorre dentro do algoritmo de treinamento. A cada passo adiante, pesos e ativações atravessam a Equação 2, com as escalas da Equação 3, e voltam ao domínio real, o que expõe o erro de quantização à função de perda. Como o arredondamento tem derivada nula em quase todo ponto, o gradiente atravessa esse operador pelo estimador direto: no passo para trás, $\partial \text{round}(x)/\partial x = 1$.

Na inferência em hardware, a acumulação da Equação 1 é inteira e o retorno a 8 bits usa um multiplicador inteiro e um deslocamento:

$$q_{\text{out}} = \text{clamp}\left(\frac{\text{acc} \cdot M + 2^{S-1}}{2^S}, -128, 127\right), \quad M = \text{round}\left(\frac{s_w s_x}{s_{\text{out}}} \cdot 2^S\right) \quad (4)$$

com $S = 16$ bits de parte fracionária; o termo 2^{S-1} arredonda ao inteiro mais próximo em vez de truncar. Não há ponto flutuante no caminho de dados. A Equação 4 é idêntica à do modelo de referência em software, o que permite exigir igualdade bit a bit entre os dois.

3.3 Fluxo contínuo

Na formulação por janela, cada decisão recomputa a convolução sobre toda a janela deslizante. Com janela de 1024 amostras e passo unitário, o mesmo trabalho é feito 1024 vezes.

Em fluxo, o motor não reinicia e a média global passa a ser mantida como soma corrente. A saída da camada densa é linear nessa soma, portanto é atualizada na chegada de cada ativação, subtraindo a que sai da janela e somando a que entra:

$$\text{logit}_c \ += \ w_{c,\text{ch}} \cdot (a_{\text{novo}} - a_{\text{antigo}}) \quad (5)$$

À mesma taxa de decisão, o custo cai de 1 040 384 para 8 128 operações por decisão, uma redução de $128\times$.

A formulação impõe duas condições. A normalização precisa ser por condição de operação: normalizar por janela divide pelo desvio da própria janela e quebra a invariância a deslocamento que permite reaproveitar a convolução entre janelas vizinhas. E a cabeça precisa ser por média global, pois em uma cabeça densa achatada o peso depende da posição dentro da janela e não existe forma incremental.

4 Metodologia

4.1 Rede, dados e treinamento

A rede tem L camadas convolucionais, F filtros na primeira, dobrando a cada camada seguinte, kernel de tamanho K , *pooling* de fator 2 entre camadas e cabeça por média global seguida de uma densa de quatro saídas.

Os dados vêm do *Case Western Reserve University Bearing Data Center*: sinais de acelerômetro de um rolamento de motor elétrico, classificados em normal, falha na pista interna, falha na esfera e falha na pista externa. Os sinais são reamostrados para 12 kHz e recortados em janelas de 1024 amostras com passo de 512. A avaliação separa uma condição de carga inteira, de 3 hp, ausente do treinamento.

O treinamento tem duas fases. A primeira usa ponto flutuante por até 120 épocas, com otimizador Adam, taxa inicial de 3×10^{-3} , aquecimento de 5 épocas, decaimento por cosseno, *weight decay* de 10^{-4} , suavização de rótulo de 0,1, *mixup* com $\alpha = 0,2$ e parada antecipada. A segunda roda 15 épocas com a quantização ativa e taxa reduzida para 10^{-4} . A janela de entrada é convertida para inteiro de 8 bits ainda no treinamento, de modo que o modelo entregue ao fluxo esteja em int8, com as escalas fixadas.

4.2 Modelo da arquitetura

A geometria da rede fixa, para cada camada l , o número de canais de entrada NIF_l , de filtros NOF_l e o comprimento LEN_l . O trabalho da camada, em multiplicações e acumulações, é

$$W_l = \text{NOF}_l \cdot \text{NIF}_l \cdot K \cdot \text{LEN}_l \quad (6)$$

O circuito de cada camada é descrito por três fatores de desdobramento: POF_l filtros, POX_l posições e PK_l taps calculados em paralelo, que determinam as vias de multiplicação instanciadas e os ciclos para concluir o trabalho:

$$D_l = \text{POF}_l \cdot \text{POX}_l \cdot \text{PK}_l, \quad C_l = \frac{W_l}{D_l} \quad (7)$$

As camadas operam encadeadas, então o intervalo de iniciação do conjunto é o maior intervalo individual:

$$\Pi = \max_l C_l \quad (8)$$

A repartição resolve, para um alvo Π^* e um orçamento $D_{\max}$ de multiplicadores, o problema da Equação 9:

$$\min \sum_l D_l \quad \text{sujeito a} \quad C_l \leq \Pi^* \quad \forall l, \quad \sum_l D_l \leq D_{\max} \quad (9)$$

A qualidade do resultado é a fração dos ciclos-multiplicador que realizam trabalho útil:

$$\eta = \frac{\sum_l W_l}{D_{\text{total}} \cdot \Pi}, \quad \eta \leq 1 \quad (10)$$

A largura do acumulador de cada camada decorre do pior caso da Equação 1 com operandos de 8 bits, acrescida de um bit de sinal e um para a polarização:

$$A_l = \lceil \log_2(\text{NIF}_l \cdot K \cdot 127^2) \rceil + 2 \quad (11)$$

Em fluxo contínuo, o intervalo entre decisões é o intervalo de iniciação escalado pela razão entre a taxa de decisão e o comprimento da janela:

$$\Delta = \frac{\Pi \cdot \rho}{\text{LEN}}, \quad \rho = \rho_{\text{pool}} \cdot \rho_{\text{dobra}} \quad (12)$$

onde ρ_{pool} é o produto dos fatores de *pooling* da rede e ρ_{dobra} vem do agrupamento de posições na última camada.

4.3 Geração do RTL

O planejador enumera, para cada camada, as repartições viáveis, descarta as dominadas e combina as sobreviventes. Como as camadas só interagem pelo II da Equação 8, fixado um alvo cada camada torna-se um problema independente, e a fronteira de Pareto sai ao custo de um único ponto [4, 10].

Algoritmo 1 — repartição dos multiplicadores

Entrada: geometria das camadas, orçamento $D_{\max}$ de multiplicadores

```
para cada camada l:
    S[l] <- {}
    para PK divisor de K:
        para POX potencia de 2 ate LEN[l]:
            para POF potencia de 2 ate NOF[l]:
                se a combinacao e' legal para a camada:
                    D <- POF*POX*PK + vias de requantizacao
                    C <- W[l] / (POF*POX*PK)
                    S[l] <- S[l] + {(D, C, BRAM, FF)}
    S[l] <- nao_dominados(S[l])

F <- {}
para cada alvo II* presente em algum S[l]:
    V[l] <- { p em S[l] : C(p) <= II* }, para todo l
    se algum V[l] e' vazio: proximo alvo
    e <- combinacao de V[1]..V[L] que minimiza soma(D)
    se soma(D) <= D_max: F <- F + {(II*, e)}
```

devolve o ponto de F com menor II

Escolhida a repartição, o emissor percorre as camadas e escreve o Verilog. A cadeia tem sempre a mesma composição: um motor por camada, um elo entre camadas consecutivas e uma cabeça ao final. Número de filtros, tamanho de kernel e tipo de *pooling* entram como valores de parâmetro na instanciação do motor, sendo o *pooling* representado por dois inteiros, POOL e POOL_AVG, com POOL= 1 para sua ausência.

Algoritmo 2 — emissão do Verilog

Entrada: plano com, por camada, (NIF, NOF, K, LEN, POOL, POOL_AVG, POF, POX, PK, NRQU, ACC_W)

T <- cabecalho do modulo acelerador_gen e suas portas

```
para l = 1 ate L:
    T <- T + instancia de conv1d_engine com os parametros da camada l
    T <- T + instancia de carga_pipe (porta de escrita de peso)
    se l < L:
```

```

T <- T + instancia de agrupador
T <- T + instancia de fifo_sinc, largura NOF[1]*8
liga saida(1) -> fila -> entrada(l+1)

T <- T + instancia de cabeca_gap_fluxo
T <- T + mapa das memorias de peso e polarizacao

escreve acelerador_gen.v, plano.json e params.tcl

```

A cabeça, que acumula pela Equação 5, impõe o único limite de faixa. Configurações com quatro camadas, 32 filtros iniciais e sem *pooling* acumulariam 262 144 termos e exigiriam 33 bits contra os 32 do caminho de dados; nesses casos o gerador recusa a configuração informando o número.

A Figura 1 mostra o caminho de dados gerado para a implementação de referência.

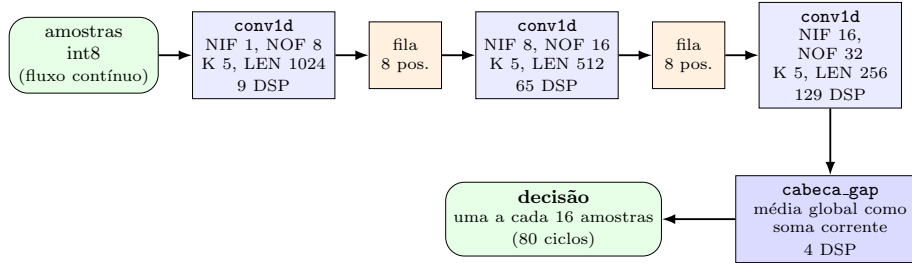


Figure 1: Caminho de dados gerado. Os motores são instâncias do mesmo módulo Verilog, com valores de parâmetro diferentes. As filas entre camadas têm 8 posições e servem ao sincronismo *valid/ready*.

4.4 Pipeline

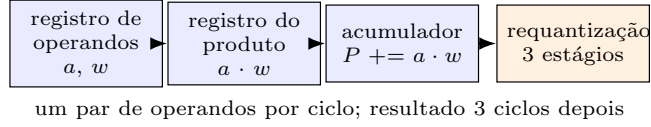
O paralelismo temporal existe em dois níveis, ilustrados na Figura 2.

Dentro da via de multiplicação. Cada via é um bloco DSP48E1 com três estágios de registro. No primeiro, os registradores de entrada capturam a amostra e o peso. No segundo, o registrador do multiplicador guarda o produto. No terceiro, o registrador de saída acumula esse produto sobre o valor anterior, ou sobre a polarização quando o sinal de carga indica o início de um novo grupo. A via aceita um par de operandos por ciclo e entrega o resultado três ciclos depois, de modo que a latência não reduz a vazão. A diretiva `use_dsp = "yes"` garante o mapeamento para o bloco dedicado em vez de lógica. Após a acumulação, a requantização da Equação 4 acrescenta mais três estágios, com o produto pelo multiplicador inteiro e o deslocamento com saturação.

Entre as camadas. Cada camada escreve em uma fila e lê da anterior com sinalização *valid* e *ready*. A camada seguinte consome em rajadas, e a fila desacopla os dois ritmos; a contrapressão impede perda de amostra quando o consumidor não pode aceitar. Como cada motor enxerga apenas filas nas extremidades, a profundidade da rede não altera o esquema de sincronismo.

Em regime, todas as camadas operam ao mesmo tempo sobre trechos diferentes do sinal: enquanto a primeira processa a amostra mais recente, a última trabalha sobre ativações originadas de amostras anteriores. Uma decisão sai a cada Δ ciclos da Equação 12, e não ao fim de um cálculo completo. O enchimento do cano é pago uma única vez, no início do fluxo.

(a) via de multiplicação (DSP48E1)



(b) camadas em regime

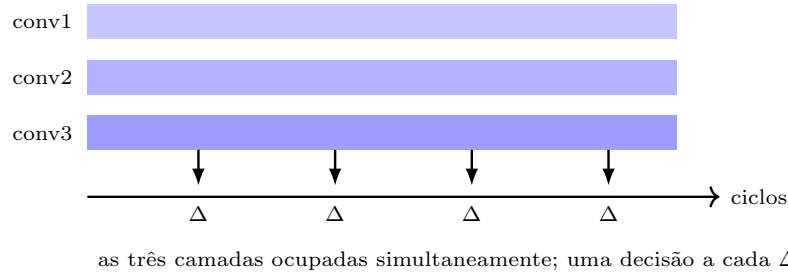


Figure 2: Pipeline em dois níveis. Em (a), os estágios de registro dentro de uma via de multiplicação. Em (b), as camadas em regime, todas ativas ao mesmo tempo sobre trechos diferentes do sinal, com uma decisão saindo a cada Δ ciclos.

4.5 Fluxo de execução

O sistema é operado por dois comandos:

```
python3 treinar.py --camadas L --filtros F --kernel K --pooling P
python3 acelerar.py --rede <nome-da-rede> --fluxo --clock 125
```

com $L \in \{2, 3, 4\}$, $F \in \{8, 16, 32\}$, $K \in \{3, 5, 7\}$ e $P \in \{\text{max}, \text{avg}, \text{none}\}$.

A Figura 3 separa as duas etapas. A quantização pertence ao treinamento, e o fluxo automatizado recebe a rede já em int8, executando sem intervenção manual até as medidas na placa.

4.6 Execução na placa

A placa é a da Figura 4. O *bitstream* gerado pelo Vivado e a aplicação em C são reunidos em uma imagem PetaLinux, carregada por JTAG. O ARM Cortex-A9 executa Linux e atua como orquestrador: carrega os pesos por AXI-Lite, alimenta as amostras por DMA e colhe as decisões de uma fila preenchida pela lógica programável, que executa a aritmética. O processador roda com prioridade de tempo real, memória travada, núcleo dedicado e interrupções deslocadas para o outro núcleo. Todo o armazenamento fica em BRAM, e o caminho de inferência dispensa DDR externa.

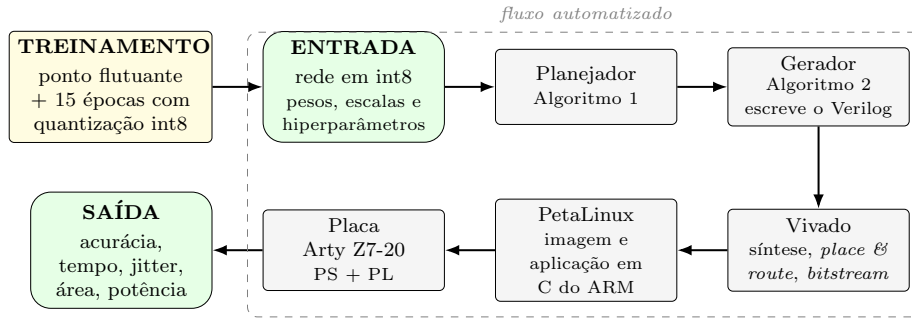


Figure 3: Etapas do sistema. A quantização faz parte do algoritmo de treinamento; o fluxo automatizado recebe a rede em inteiro de 8 bits e devolve as medidas da placa.

Cada decisão sai numerada pelo hardware, com quatro bits de sequência junto da classe, e o processador confere a contiguidade, de modo que uma decisão perdida entre a fila e o processador seja detectada.

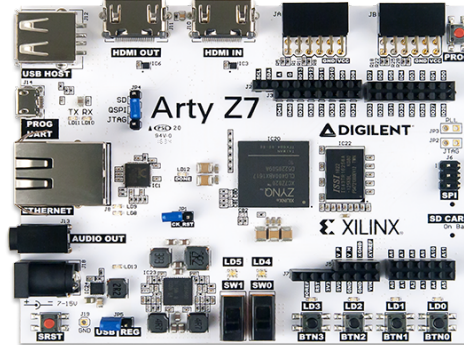


Figure 4: Plataforma de medição: Digilent Arty Z7-20, SoC Zynq-7000 XC7Z020-1. Fonte da imagem: Digilent.

5 Resultados e Discussão

5.1 Cobertura do domínio

O fluxo foi exercitado em topologias que percorrem todos os valores de cada eixo, com $L \in \{2, 3, 4\}$, $F \in \{8, 16, 32\}$, $K \in \{3, 5, 7\}$ e os três tipos de *pooling*. Cada uma passou pela quantização, pela repartição, pela emissão do Verilog e pela verificação de exatidão. A Tabela 1 traz os circuitos derivados.

Topologia	DSP	BRAM	Ciclos/amostra	η	Δ (amostras)
L2_F08_K3_Pmax	168	1	1,5	0,921	16
L2_F16_K5_Pavg	142	17	10,0	0,980	8
L2_F32_K7_Pnone	136	35	112,0	0,973	2
L3_F08_K5_Pmax	207	17	5,0	0,982	16
L3_F08_K5_Pnone	174	17	20,0	0,968	4
L3_F16_K7_Pmax	203	37	28,0	0,971	8
L3_F32_K3_Pavg	201	39	48,0	0,972	8
L4_F08_K7_Pavg	122	21	28,0	0,939	16
L4_F32_K5_Pmax	121	66	320,0	0,931	16

Table 1: Circuitos derivados ao longo do domínio. η é a eficiência da Equação 10 e Δ o intervalo entre decisões da Equação 12.

A verificação de exatidão ocorre em dois níveis. No nível de camada, 71 configurações de (NIF, NOF, K , LEN, POX, POF, PK) foram comparadas com a referência em inteiro, com bolha na entrada e contrapressão na saída, todas bit-exatas. No nível de rede, cada topologia foi comparada com a cadeia completa em inteiro sobre um fluxo contínuo de amostras, verificando também o intervalo entre decisões contra a Equação 12.

O intervalo de η observado, de 0,921 a 0,982, decorre do custo fixo por camada: as vias de requantização e a cabeça não escalam com o arranjo de multiplicadores, e seu peso relativo cresce nas topologias que usam menos DSP.

5.2 Implementação de referência

A Tabela 2 traz a repartição para a implementação de referência. As três camadas concluem nos mesmos 5 120 ciclos, e as convoluções somam 1 024 000 operações ocupando 200 vias durante esses ciclos. A síntese usou Vivado 2025.2 sobre o XC7Z020-1, com diretivas padrão e sem semente aleatória.

O circuito usa 207 blocos DSP, sendo 204 vias de multiplicação e três multiplicadores de requantização, um por camada convolucional. Pela Equação 12, com $\rho = 16$, o intervalo entre decisões previsto é de 80 ciclos.

Camada	NIF	NOF	LEN	POF	POX	W_l	C_l	A_l
conv1	1	8	1024	4	2	40 960	5 120	19
conv2	8	16	512	16	4	327 680	5 120	22
conv3	16	32	256	32	4	655 360	5 120	23
cabeça	—	—	—	—	—	16 384	—	27

Table 2: Repartição da implementação de referência. W_l e C_l vêm das Equações 6 e 7; A_l , da Equação 11, contra os 48 bits do acumulador do DSP48E1.

Grandeza	Valor
Decisões avaliadas	311 163
Concordância com o modelo de referência	100,00%
Acurácia sobre janelas completas	90,12%
Decisões perdidas por fila cheia	0
Saltos na sequência	0
Estouros de acumulador	0

Table 3: Medido na placa, sobre o conjunto de teste completo.

Exatidão

A Tabela 3 reúne as medidas. O circuito reproduz o modelo quantizado decisão por decisão. A acurácia de 90,12% é propriedade do modelo: fica cerca de 5 pontos abaixo da obtida com normalização por janela, diferença que corresponde ao custo de viabilizar o fluxo contínuo.

Determinismo

O intervalo entre decisões é contado pela lógica programável, no ciclo em que a decisão sai. Um segundo contador registra os ciclos em que o motor aceitaria uma amostra e não havia nenhuma disponível. Os dois separam as origens possíveis de um intervalo maior que o previsto pela Equação 12: havendo parada por entrada vazia, a origem é o alimentador; não havendo, seria o circuito.

A primeira medida deu 80 a 642 ciclos, com 0,41% dos ciclos em parada por entrada vazia. O mínimo coincidia com o previsto, o que localizou a origem no alimentador: o processador preenchia o próximo lote e colhia as decisões depois de aguardar a transferência DMA, e a fila de entrada se esvaziava nesse intervalo. A correção foi feita no software do processador, com dois *buffers* alternados e a cópia sobreposta à transferência em curso.

A Tabela 4 compara as duas medidas. As paradas remanescentes ocorrem enquanto a fila de entrada mantém 2048 amostras de reserva, e por isso não alcançam o motor. O intervalo final é o previsto, sem exceção em 311 162 observações.

	Antes	Depois
Intervalo entre decisões (ciclos)	80–642	80–80
Jitter (ciclos)	562	0
Intervalos medidos	311 162	311 162
Ciclos em parada por entrada vazia	0,41%	0,40%

Table 4: Intervalo entre decisões antes e depois da correção no alimentador.

Previsão e medida

Grandeza	Previsto	Medido
Blocos DSP	207	207
Blocos BRAM36	17	17
Ciclos por amostra	5,00	5,02
η (operações por DSP-ciclo)	0,982	0,978
Intervalo entre decisões (ciclos)	80	80–80

Table 5: Previsão contra a síntese e o silício. O excesso de 0,4% em ciclos por amostra é coberto pelas paradas de entrada medidas.

Como mostra a Tabela 5, a previsão de DSP e BRAM coincide com a síntese, e o fluxo recusa a rodada caso diverjam. LUT e *flip-flops* são medidos e reportados, mas não previstos, já que o planejador não constrói modelo de área lógica.

Área, frequência e consumo

A Tabela 6 traz a ocupação, o fechamento temporal e o consumo. A análise de tempo aplica incerteza de relógio de 5% do período, conforme UG903, de modo que a ferramenta otimize contra essa margem.

Comparação com o processador da mesma placa

A linha de base é a mesma inferência, também em fluxo contínuo, executada no ARM Cortex-A9 do próprio SoC, em C compilado com `-O3` e sob as mesmas condições de tempo real. Manter os dois em fluxo mantém a comparação em regime equivalente. O resultado está na Tabela 7.

Grandeza	Valor
LUT	14 902 (28,0%)
Flip-flops	18 503 (17,4%)
BRAM36	17 (12,1%)
DSP48E1	207 (94,1%)
Frequência fechada	125,0 MHz
Caminhos em falha (<i>setup</i> / <i>hold</i>)	0 / 0
Avisos críticos do projeto gerado	0
Violações de metodologia	0
Potência total (estimada)	2,28 W
Potência dinâmica da PL (estimada)	0,855 W
Energia por decisão na PL (estimada)	0,549 μ J

Table 6: Sistema completo em XC7Z020-1, estado roteado. Os valores de potência são estimativa da ferramenta, sem vetores de atividade.

	ARM em fluxo	Lógica programável
Amostras por segundo	85 412	24 906 880
Ganho no cômputo	—	291,6×
Jitter entre decisões	—	0 ciclos

Table 7: Regime equivalente, fluxo contra fluxo. O código do ARM não usa NEON escrito à mão nem biblioteca otimizada.

6 Conclusão

O trabalho entrega um procedimento que deriva, a partir de uma rede convolucional 1D treinada e quantizada, o circuito dedicado que a executa. Topologias percorrendo todos os valores dos eixos foram geradas e verificadas por comparação bit-exata, e a implementação de referência foi caracterizada em uma Arty Z7-20.

Nessa implementação, o circuito reproduz o modelo quantizado em 311 163 decisões; o intervalo entre decisões é o previsto pela Equação 12, com jitter nulo em 311 162 intervalos contados pela lógica programável; a previsão de DSP e BRAM coincide com a síntese; a eficiência da Equação 10 é de 0,978 operações por DSP-ciclo contra um teto de 1,0; e o ganho sobre o processador da própria placa em regime equivalente é de $291,6\times$.

Como trabalho futuro, a plataforma serve de base para o aprendizado incremental previsto na proposta do projeto, aproveitando o arranjo de multiplicadores e a porta de escrita de peso já instanciados. Ficam também a medição de potência com vetores de atividade e o levantamento de mais configurações na placa.

References

- [1] Wu, H., Judd, P., Zhang, X., Isaev, M., & Micikevicius, P. (2020). *Integer Quantization for Deep Learning Inference: Principles and Empirical Evaluation*. arXiv preprint arXiv:2004.09602.
- [2] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., ... & Adam, H. (2018). *Quantization and training of neural networks for efficient integer-arithmetic-only inference*. In Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR) (pp. 2704-2713).
- [3] Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., ... & Wu, H. (2018). *Mixed precision training*. International Conference on Learning Representations (ICLR).
- [4] Zhang, C., Li, P., Sun, G., Guan, Y., Xiao, B., & Cong, J. (2015). *Optimizing FPGA-based Accelerator Design for Deep Convolutional Neural Networks*. In Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA) (pp. 161-170).
- [5] Motamedi, M., Ghiasi, S., & Fong, D. (2016). *Design Space Exploration of FPGA Accelerators for Convolutional Neural Networks*. In Design, Automation & Test in Europe Conference & Exhibition (DATE) (pp. 1156-1159).
- [6] Venieris, S. I., & Bouganis, C. S. (2018). *Toolflows for Mapping Convolutional Neural Networks on FPGAs: A Survey and Future Directions*. ACM Computing Surveys (CSUR), 51(3), 1-39.
- [7] Qiu, J., Wang, J., Yao, S., Guo, K., Li, B., Zhou, E., ... & Yao, P. (2016). *Going Deeper with Embedded FPGA Platform for Convolutional Neural Network*. In Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA) (pp. 26-35).
- [8] Guo, K., Zeng, S., Yu, J., Wang, Y., & Yang, H. (2017). *Angel-Eye: A Complete Design Flow for Mapping CNN onto Embedded FPGA*. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD), 37(1), 35-47.
- [9] Moreau, T., Chen, T., Jiang, Z., Ceze, L., Guestrin, C., & Krishnamurthy, A. (2019). *VTA: An Open Hardware-Software Stack for Deep Learning*. IEEE Micro, 39(6), 38-46.
- [10] Ma, Y., Cao, Y., Vrudhula, S., & Seo, J. S. (2017). *Optimizing Loop Operation and Dataflow in FPGA Acceleration of Deep Convolutional Neural Networks*. In Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA) (pp. 45-54).

- [11] Sharma, H., Park, J., Mahajan, D., Amaro, E., Kim, J. K., Shao, C., ... & Esmaeilzadeh, H. (2016). *DNNWeaver: From High-Level Deep Network Models to FPGA Acceleration*. In 2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO) (pp. 1-12).
- [12] Wei, X., Yu, C. H., Zhang, P., Chen, Y., Wang, Y., Hu, H., ... & Cong, J. (2017). *Automated Systolic Array Architecture Synthesis for High Throughput CNN Inference on FPGAs*. In Proceedings of the 54th Annual Design Automation Conference (DAC) (pp. 1-6).
- [13] Sze, V., Chen, Y. H., Yang, T. J., & Emer, J. S. (2017). *Efficient Processing of Deep Neural Networks: A Tutorial and Survey*. Proceedings of the IEEE, 105(12), 2295-2329.
- [14] Umuroglu, Y., Fraser, N. J., Gambardella, G., Blott, M., Leong, P., Jahre, M., & Vissers, K. (2017). *FINN: A Framework for Fast, Scalable Binarized Neural Network Inference*. In Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA) (pp. 65-74).
- [15] Gysel, P., Motamedi, M., & Ghiasi, S. (2016). *Hardware-oriented Approximation of Convolutional Neural Networks*. International Conference on Learning Representations (ICLR) Workshop.